

Full-Stack Developer – Technical Case Study

Stack: PostgreSQL + Express.js + React.js + Node.js

Deadline: 48 Hours

Expected Effort: Approximately 10–14 Hours

Business Scenario

A manufacturing and supply company sells industrial products to business customers.

Build a small ERP application covering the following workflow:

Customer Enquiry → Quotation → Sales Order → Inventory Reservation → Dispatch

The objective is not to recreate a complete ERP.

We want to evaluate whether you can independently understand and implement a real business workflow using the **PERN stack**.

1. Authentication & Roles

Implement authentication with:

ADMIN

- View all records
- Manage inventory
- Confirm Sales Orders
- Process dispatch

SALES USER

- Create customers/enquiries
- Create quotations
- Convert accepted quotations into Sales Orders
- View inventory availability

Requirements:

- JWT authentication
- Password hashing
- Protected APIs
- Backend role authorization

Frontend-only role restrictions will not be considered sufficient.

2. Customer Enquiry

Sales User should be able to create an enquiry.

Customer Details

- Company Name
- Contact Person
- Mobile
- Email
- City

Enquiry Details

- Enquiry Number
- Customer
- Enquiry Date
- Required Date
- Products
- Quantity
- Notes
- Status

An enquiry may contain **multiple products**.

Example:

Customer: ABC Engineering Pvt. Ltd.

Product	Quantity
Industrial Product A	100
Industrial Product B	40
Industrial Product C	200

Statuses:

NEW → QUOTED → WON / LOST

3. Product & Inventory

Create a basic Product Master.

Each product should have:

- Product Code

- Product Name
- Category
- Unit
- Base Price

Seed at least **6 industrial products** with realistic sample data.

Inventory should maintain:

- Product
- Physical Quantity
- Reserved Quantity

The system should calculate:

Available Quantity = Physical Quantity – Reserved Quantity

Example:

Physical = 200

Reserved = 60

Available = **140**

The system must prevent:

- Negative quantities
 - Reservation beyond available stock
 - Invalid inventory updates
-

4. Quotation

Sales User can create a quotation against an enquiry.

Quotation contains:

- Quotation Number
- Enquiry Reference
- Customer
- Product(s)
- Quantity
- Unit Price
- Discount %
- GST %
- Line Amount
- Grand Total
- Valid Until
- Status

Statuses:

DRAFT → SENT → ACCEPTED / REJECTED

Calculation

For each item:

Base Amount = Quantity × Unit Price

Apply discount and GST to calculate the final amount.

Final quotation amount must be calculated or validated by the backend.

Do not blindly accept a final amount sent by React.

5. Quotation → Sales Order

An **ACCEPTED quotation** can be converted into a Sales Order.

Sales Order should contain:

- Order Number
- Customer
- Quotation Reference
- Order Date
- Products
- Quantity
- Total Amount
- Status

Statuses:

PENDING → CONFIRMED → DISPATCHED → CANCELLED

Rules:

- DRAFT quotation cannot create an order.
- REJECTED quotation cannot create an order.
- One quotation should not accidentally generate multiple Sales Orders.

The relationship should remain traceable:

Customer → Enquiry → Quotation → Sales Order

6. Inventory Reservation

When an Admin confirms the Sales Order, inventory must be checked.

Example:

Product requirement = **80**

Inventory:

Physical = 100

Reserved = 30

Available = 70

The order **cannot be confirmed for 80 units** because only 70 are available.

If requirement = 60:

Before:

Physical = 100

Reserved = 30

Available = 70

After confirmation:

Physical = 100

Reserved = 90

Available = 10

Physical inventory should **not decrease during reservation**.

Important Backend Challenge

Consider:

Available inventory = **100**

Two requests arrive almost simultaneously:

User A → Reserve 80

User B → Reserve 50

Your application must ensure that both reservations cannot succeed.

This must be solved at the **backend/database level** using an appropriate technical approach.

A React-side stock check alone is not sufficient.

7. Dispatch

Admin can dispatch a confirmed Sales Order.

Dispatch should contain:

- Dispatch Number
- Sales Order
- Dispatch Date
- Product(s)
- Quantity
- Vehicle Number
- Driver Name

When stock is dispatched:

Physical Quantity decreases

AND

Reserved Quantity decreases

Example:

Before:

Physical = 100

Reserved = 60

Available = 40

Dispatch = 60

After:

Physical = 40

Reserved = 0

Available = 40

The system must prevent:

- Dispatch beyond reserved quantity
- Duplicate dispatch of the same quantity
- Dispatch of a cancelled order

Required Screens

Build only these **4 main screens**:

1. Login

2. Enquiries

Create and view enquiries.

3. Quotations

Create quotations and Accept/Reject.

4. Sales Orders

View orders, stock availability, confirm/reserve stock and process dispatch.

Inventory availability may be displayed within the Sales Order screen or as a small separate section.

Do not spend excessive time on UI design.

A clean, functional and responsive interface is sufficient.

Database Expectations

Use PostgreSQL with proper relational modelling.

Expected entities may include:

- users
- customers
- products
- inventory
- enquiries
- enquiry_items
- quotations
- quotation_items
- sales_orders
- sales_order_items
- dispatches

You may design the schema differently if you can explain your approach.

We will evaluate:

- Foreign keys
- Relationships
- Constraints
- Unique fields
- Transactions
- Data consistency

Do not store the entire business workflow inside JSON fields simply to avoid relational database design.

API Expectations

Create clean REST APIs.

Examples:

POST /auth/login

POST /enquiries

GET /enquiries

POST /quotations

PATCH /quotations/:id/status

POST /quotations/:id/convert

GET /sales-orders

POST /sales-orders/:id/confirm

POST /sales-orders/:id/dispatch

You are free to improve the API design.

Mandatory Tests

Implement at least **5 automated tests**.

Test 1

Quotation total is calculated correctly.

Test 2

Rejected/Draft quotation cannot create a Sales Order.

Test 3

Same quotation cannot generate duplicate Sales Orders.

Test 4

Cannot reserve more than available inventory.

Test 5

Unauthorized user cannot perform a restricted operation.

Bonus

Test simultaneous inventory reservations.

Technical Requirements

Mandatory:

- React.js
- Node.js
- Express.js
- PostgreSQL
- REST APIs
- JWT Authentication
- Backend RBAC
- Validation
- Error handling
- Database relationships
- Transactions where required
- Environment-based configuration

ORM is your choice.

Examples:

Prisma / Sequelize / Drizzle / Knex

Submission Requirements

Submit within **48 hours**.

1. Git Repository

Complete source code.

Repository should demonstrate reasonable development history.

Do not submit the complete project as one final commit.

2. README

Include:

- Tech stack
- Project setup
- Database setup
- Environment variables
- Migration/seed instructions
- How to run frontend/backend
- How to run tests
- Test login credentials

3. Database Schema / ER Diagram

4. API Documentation

Postman / Swagger / equivalent.

5. Demo Video

Maximum **5 minutes**.

Demonstrate:

Login → Enquiry → Quotation → Sales Order → Reservation → Dispatch

AI Usage

AI tools are allowed.

However, you must understand every major part of your submission.

Shortlisted candidates may be asked to:

- Explain their PostgreSQL schema
- Explain an API
- Explain authentication/RBAC
- Modify business logic
- Debug an issue
- Make a small database change

Candidates unable to modify or explain their own submission will not qualify.

Live Verification Round

Shortlisted candidates will receive **one unannounced change**.

For example:

Add **DAMAGED** stock.

Inventory should now maintain:

Physical = 100

Reserved = 20

Damaged = 10

Therefore:

Available = 100 – 20 – 10 = 70

Update the:

- Database
- Backend logic
- Relevant API
- Frontend display

Another possible change:

Allow a confirmed Sales Order to be cancelled and correctly release its reserved inventory.

Expected live implementation/explanation time:

20-30 minutes

What We Actually Evaluate

We are not looking for the candidate who builds the most attractive dashboard.

We are looking for someone who can correctly understand and implement:

Enquiry → Quotation → Order → Stock Reservation → Dispatch

A simple frontend with strong backend logic and PostgreSQL design will score significantly higher than a beautiful frontend with incorrect business logic.

Deadline: Maximum 48 Hours from assignment receipt.